BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Abstraction and Decomposition

Name _____ **Class** _____ **Date** _____ **Time** _____ **Teacher** _____

Do Now - Four retrieval questions

BLACK PEN: answer four questions before checking the answers.

1. What makes a technology issue ethical?

2. Which technology does a USB flash drive use?

A) Solid-state
B) Optical
C) Magnetic
D) Volatile RAM

3. "Compression must increase every file's size." Correct this. Explain why.

4. A website thumbnail may tolerate a small reduction in detail. "Lossy compression guarantees an identical original after decompression." Correct this. What could happen?

Today's objectives

Define abstraction and decomposition accurately. Break a complex problem into smaller tasks.

Key vocabulary - Read, cover, recall

Computational thinking
Techniques for representing and solving problems logically.

Decomposition
Breaking a complex problem into manageable smaller parts.

Abstraction
Focusing on relevant details and ignoring unnecessary details.

Subproblem
Smaller task forming part of a larger problem.

Write one key term and its meaning in your own words:

Do Now	I Do	We Do	You Do	Review & Improve
Recall previous learning	Read new knowledge	Practise with guidance	Work independently	Check my understanding

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Learn and practise

New learning - Key ideas

Computational thinking	Computational thinking helps define a problem so a computer solution can be designed. OCR J277 specifies abstraction, decomposition and algorithmic thinking.
Decomposition	Split a large problem into manageable subproblems, such as input, calculations, checking and output. Each subproblem can be developed and tested more easily.
Abstraction	Identify essential details and ignore information not needed to solve the problem. A simplified map may show roads and stations but omit the colour of nearby buildings.
Use both together	First identify the required result; break the job into parts; retain information each part actually needs. Abstraction does not mean deleting important data.

I Do - One worked example

A club wants a program to calculate the cost of booking sports courts. Explain how decomposition and abstraction would help create the program. [4 marks]

1. Define the task: calculate and display a booking cost.

2. Decompose into input duration, get hourly price, calculate total and display it [1], making development manageable [1].

3. Abstract away irrelevant details such as court wall colour [1], keeping duration and rate needed to calculate cost [1].

4. Use specific examples from the booking problem.

We Do - Guided questions

BLACK PEN: use the model, then explain your answer.

1. A cinema needs an online ticket-pricing program. Explain two ways decomposition helps the programmer. [4 marks]

2. A bus-route planner uses distances and stops but ignores building colours. Explain how abstraction helps. [4 marks]

3. A school lunch app totals several purchases. Explain how decomposition and abstraction work together. [4 marks]

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Think and write

Code First - Predict > Write > Test > Improve

YOUR TASK: Write a line to show the cost of three storage cards.

```
unitPrice = 5
quantity = 3
```

BLACK PEN: predict or complete the code; test, then improve.

For extended answers, continue on lined paper if needed.

You Do - Independent exam questions

Q2. Define abstraction in computational thinking. [2 marks]

Q3. A weather app predicts whether to show an umbrella warning. Explain how decomposition and abstraction help design it. [4 marks]

Q4. Explain abstraction and decomposition using an example of a simple school timetable program. [6 marks]

Exit Ticket - True or False?

Circle T or F before reviewing Page 4.

1. Abstraction removes information that is irrelevant to the problem while keeping useful details. ☐ T ☐ F
2. Computational thinking helps define a problem so a computer solution can be designed ☐ T ☐ F
3. Split a large problem into manageable subproblems, such as input, calculations, checking and output ☐ T ☐ F

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Review & improve

GREEN PEN: tick correct marking points, then correct errors.

Choose your hardest question to rewrite in full below.

Do Now - Answers

Q1 It concerns fairness, responsibility or what ought to be done, even if an action is legal. ☐

Q2 A - Solid-state ☐

Q3 Reducing file size can reduce storage requirements and transmission time. ☐

Q4 Lossy compression removes information, so the exact original may be lost. A thumbnail can lose some detail; source code must be restored exactly. ☐

Code First - Model

```
unitPrice = 5
quantity = 3
print(unitPrice * quantity)
```

Output: 15

We Do - Check the model

1. Break it into input age/ticket type, calculate prices and output total [1], making each subproblem manageable [1]. Each component can be tested separately [1], helping locate errors [1]. ☐

2. Abstraction keeps distances and stops needed for route calculation [1], so the model contains relevant features [1]. It ignores building colours [1] because that information does not affect the route [1]. ☐

3. Decomposition splits the task into taking item prices, adding totals and displaying payment [1], simplifying work [1]. Abstraction keeps prices and quantities [1] while ignoring irrelevant details such as a pupil's favourite colour [1]. ☐

Exit Ticket - Check

1. TRUE 2. TRUE 3. TRUE

You Do - Self-mark

Q2 Removing/ignoring irrelevant detail [1] while keeping information required to solve a problem [1]. ☐

Q3 Break the job into reading forecast, comparing rain probability and displaying advice [2]. Keep forecast/rain likelihood [1] while ignoring irrelevant information such as the user's phone case colour [1]. ☐

Q4 Decomposition splits the program into loading lesson data, finding a day and displaying the timetable [2], helping development/testing [1]. Abstraction keeps lesson time, room and subject [1] but ignores details not needed such as wall colour [1], reducing complexity without losing essential information [1]. ☐

Try the question again

1. Copy the FULL question you found most difficult:

2. In GREEN PEN, write a complete new answer: